

BA305 — Exam 1 Mock Test · Answer Key

Each question as it appears on the mock test, followed by its answer.

Part A — Concepts (5 questions)

Question 1 — Multiple choice

In scikit-learn, you fit a k-nearest-neighbors classifier with **K equal to the number of training data points**. What happens?

- (a) It always gets the right answer.
- (b) It gives a random answer.
- (c) It gives the majority vote of the whole training set.
- (d) It gives an error.

Answer — (c) It gives the majority vote of the whole training set.

With $K = n$, the "K nearest neighbors" of any new record are **all** the training points, no matter where the record sits. Every prediction is the same: the majority class of the training set. This is the naïve rule. The model ignores its input entirely, which is the extreme of underfitting. (a) is wrong: it predicts the same class for everyone, so it misses every member of the minority class. (b) is wrong: k-NN is deterministic. (d) is wrong: $K = n$ is allowed. scikit-learn raises an error only when K is **larger** than the number of training points.

Question 2 — True / False

You run a decision tree model on your data and get a mediocre **test** accuracy of 0.56. You conclude that you need a **more complex** model because your model is **overfitting**.

- True
- False

Answer — False.

Two mistakes. First, test accuracy alone cannot tell you the model is overfitting. You need to compare it to the **training** accuracy: overfitting means high training accuracy and much lower test accuracy. If both are around 0.56, the model is underfitting. Second, even if the model were overfitting, the fix is a **simpler** model (for a tree: a smaller `max_depth`, a larger `min_samples_leaf`), not a more complex one.

Question 3 — Multiple choice

Your classification model outputs probabilities, and you threshold them to get class labels. With a threshold of 0.4, your model has an **AUC of 0.87**. To increase the model's AUC, you need to:

- (a) increase the threshold.
- (b) decrease the threshold.
- (c) know the accuracy first.
- (d) none of the other options.

Answer — (d) none of the other options.

AUC is computed from the ROC curve, which sweeps **every** threshold. It does not depend on the threshold you picked, so moving the threshold up or down leaves the AUC at 0.87. Accuracy is a different metric at one threshold and is not needed to compute AUC. To raise AUC you need a model that ranks the positives above the negatives better — different features, a different model, or different settings.

Question 4 — Multiple choice

Your dataframe has 5 numerical features. You run PCA and keep 4 principal components. Their cumulative explained variance must be:

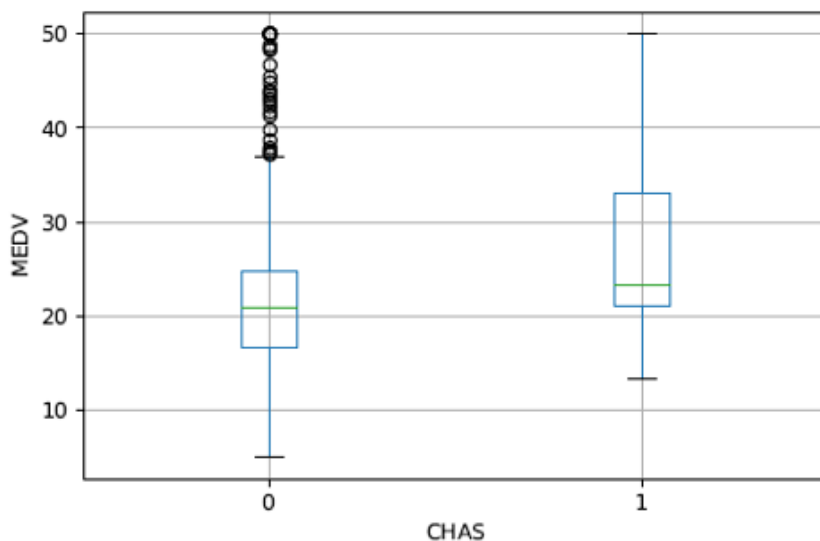
- (a) exactly 100%
- (b) exactly 80%
- (c) at most 100%
- (d) none of the other options

Answer — (c) at most 100%.

All 5 components together explain 100% of the variance. Keeping 4 drops the 5th, which explains whatever is left — usually something, so the 4 typically explain less than 100%. It can be exactly 100% when one feature is a perfect linear combination of the others (the 5th component then has zero variance). Nothing forces it to be 80%; 80% is only a common rule of thumb for how many components to keep.

Question 5 — Select all that apply

The figure below comes from the Boston Housing data. `MEDV` is the median home value (in \$1,000s) and `CHAS` is 1 if the tract borders the Charles River, 0 otherwise.



What can this figure tell us? Select **all** that apply.

- (a) `CHAS = 1` has no outliers.
- (b) At least 75% of the houses with `CHAS = 1` have a `MEDV` below 40.
- (c) The **mean** `MEDV` of `CHAS = 0` is about 20.

- (d) The highest MEDV value for CHAS = 1 is 50.

Answer — (a), (b) and (d).

- **(a) True.** Outliers are drawn as circles beyond the whiskers. CHAS = 0 has many above 37; CHAS = 1 has none.
- **(b) True.** The top of the CHAS = 1 box is the 75th percentile, about 33. So 75% of those houses are at or below 33, which is below 40.
- **(c) False.** The line inside the box is the **median** (about 21 for CHAS = 0), not the mean. A boxplot does not show the mean. With so many high outliers in CHAS = 0, the mean is pulled above the median anyway.
- **(d) True.** There are no outliers for CHAS = 1, so the top of the upper whisker is the maximum: 50.

Part B — Coding: NASA Exoplanets

You have data from the NASA Exoplanet Archive: one row per confirmed planet outside our solar system. You want to estimate the **year** a planet was discovered.

Column	Meaning
pl_name	Planet name
discoverymethod	How the planet was found (e.g., Transit, Radial Velocity)
disc_year	Year it was discovered
pl_orbper	Orbital period: days to go once around its star
pl_bmassj	Mass, in Jupiter masses (1 = as heavy as Jupiter)
sy_dist	Distance from Earth, in parsecs

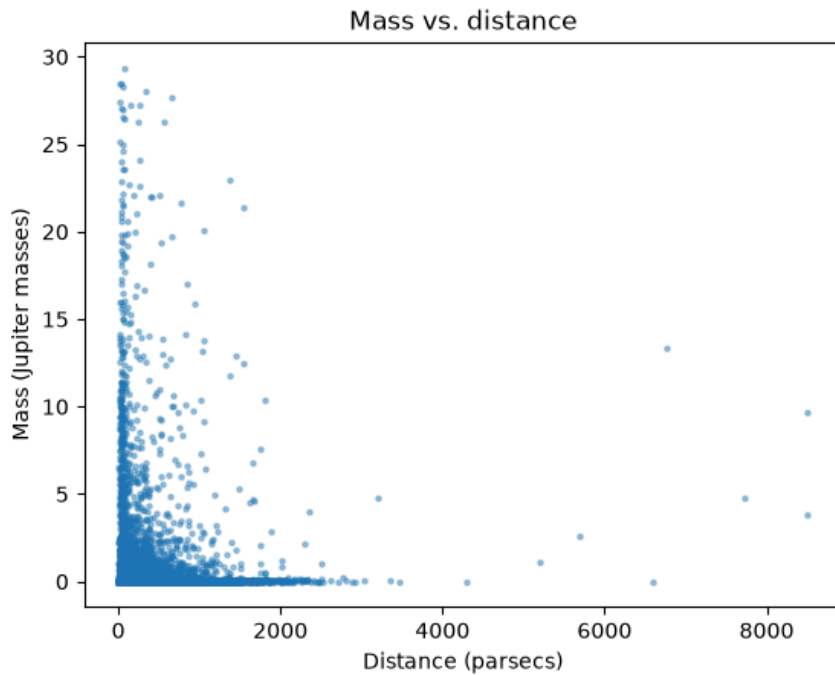
```
# Load the NASA exoplanet data into a dataframe called planets, then drop rows with missing values.
...

# Features: discovery method as dummy columns plus the three measurements; target: discovery year.
X = pd.get_dummies(planets[['discoverymethod', 'pl_orbper', 'pl_bmassj', 'sy_dist']], dtype=int)
y = planets['disc_year']
print('Features:', X.shape[1])
```

Features: 13

Question 6 — Short answer

```
# Plot mass against distance.
plt.scatter(planets['sy_dist'], planets['pl_bmassj'], s=5, alpha=0.4)
plt.xlabel('Distance (parsecs)')
plt.ylabel('Mass (Jupiter masses)')
plt.title('Mass vs. distance');
```

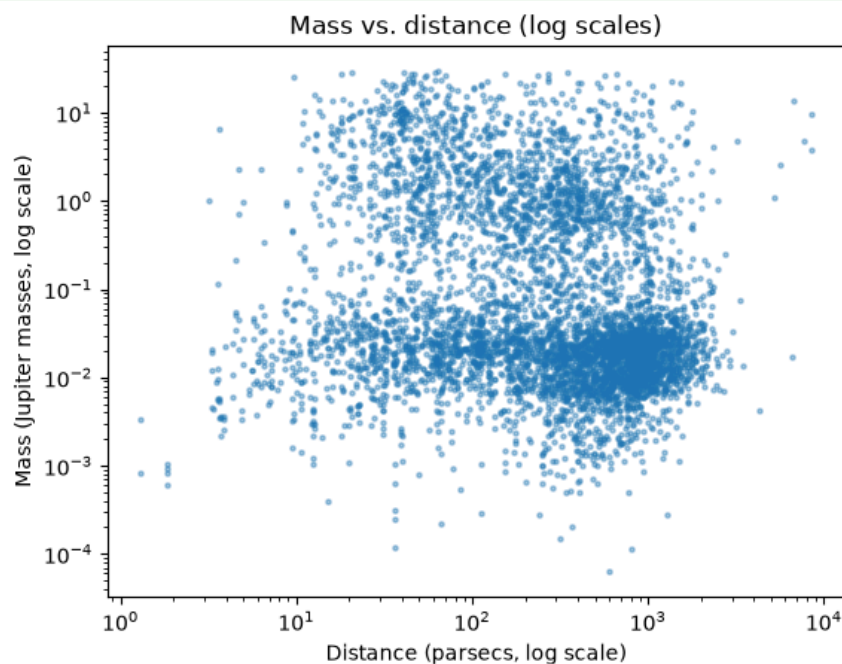


Too many data points seem crammed into one corner of this chart. How could you make the chart easier to read **without removing any data point**? Answer in 2 lines only.

Answer — Put both axes on a log scale.

Most planets are light (under 1 Jupiter mass) and fairly close (under 1,000 parsecs), while a few reach 30 Jupiter masses or 8,500 parsecs. Those few set the axis ranges, so everyone else is squeezed into the bottom-left corner. On a log scale each tick is 10 times the one before, which spreads the small values out without dropping any point:

```
plt.xscale('log')
plt.yscale('log')
```



Smaller or more transparent markers (s , alpha) help a little with overlap, but they do not fix the real problem, which is the scale.

Question 7 — Fill in the blank

```
# Scale the features, then keep fewer principal components.
X_scaled = StandardScaler().fit_transform(X)
pca_small = PCA(n_components=_____).fit(X_scaled)
pd.DataFrame({
    'component': range(1, pca_small.n_components_ + 1),
    'eigenvalue': pca_small.explained_variance_.round(2),
    'explained variance': pca_small.explained_variance_ratio_.round(3),
    'cumulative': pca_small.explained_variance_ratio_.cumsum().round(3),
})
```

	component	eigenvalue	explained variance	cumulative
0	1	2.42	0.186	0.186
1	2	1.28	0.099	0.285
2	3	1.22	0.094	0.379
3	4	1.04	0.080	0.459

Fill in the blank.

Answer — 4

```
pca_small = PCA(n_components=4).fit(X_scaled)
```

The output has 4 rows: one per component kept. Together they explain 45.9% of the variance.

Question 8 — Multiple choice

You now predict the discovery year with PCA followed by linear regression, scored with 5-fold cross-validation, trying every number of components from 1 to 13.

```
# 5-fold cross-validation: shuffle once, split into 5 parts, each part is the test set once.
folds = KFold(n_splits=5, shuffle=True, random_state=0)

# Scale -> PCA -> linear regression, refit inside every fold; try 1 to 13 components.
for k in range(1, X.shape[1] + 1):
    model = make_pipeline(StandardScaler(), PCA(n_components=k), LinearRegression())
    scores = cross_val_score(model, X, y, cv=folds, scoring='r2')
    print(f'{k} components: fold R² = {scores.round(2)}, mean = {scores.mean():.2f}')
```

1 components: fold R^2 = [-0.45 0.02 0.03 0.02 0.], mean = -0.08
 2 components: fold R^2 = [-4.09 0.02 0.03 0.02 0.01], mean = -0.80
 3 components: fold R^2 = [-3.43 0.04 0.05 0.05 0.03], mean = -0.65
 4 components: fold R^2 = [-0.07 0.04 0.06 0.06 0.03], mean = 0.02
 5 components: fold R^2 = [-0.06 0.04 0.06 0.06 0.03], mean = 0.03
 6 components: fold R^2 = [-0.06 0.04 0.06 0.07 0.03], mean = 0.03
 7 components: fold R^2 = [-0.06 0.05 0.06 0.07 0.03], mean = 0.03
 8 components: fold R^2 = [-0.06 0.05 0.06 0.07 0.03], mean = 0.03
 9 components: fold R^2 = [-0.06 0.05 0.06 0.07 0.03], mean = 0.03
 10 components: fold R^2 = [-2.17 0.05 0.06 0.07 0.03], mean = -0.39
 11 components: fold R^2 = [-4.826e+01 5.000e-02 7.000e-02 8.000e-02 4.000e-02], mean = -9.60
 12 components: fold R^2 = [-13.59 0.1 0.12 0.11 0.05], mean = -2.64
 13 components: fold R^2 = [-13.59 0.1 0.12 0.11 0.05], mean = -2.64

Which number of components leads to the best model?

- (a) 1
- (b) 5
- (c) 12
- (d) 13

Answer — (b) 5.

- **Read the mean column.** 5 through 9 components tie for the highest mean R^2 , 0.03, and 5 is the only one of them among the choices. With a tie, the fewest components is the better pick anyway: the simpler model scores just as well. (a) 1 component has a negative mean (-0.08).
- **Do not be fooled by 12–13 components.** They score best in four folds (0.05–0.12), but the first fold collapses to -13.59 and drags the mean to -2.64. One bad fold can sink a model. That is why we look at every fold, not just the best ones.
- **The first fold is negative for every k.** Something in that fold, most likely a few extreme planets (orbital periods run up to 400 million days), is far from anything the model learned in the other four folds. A negative R^2 means the model does worse than simply guessing the average year.
- **Even the best model is weak.** An R^2 of 0.03 means these features explain only about 3% of the variation in discovery year. The best number of components is the best of a poor set of models.

Question 9 — Short answer

Your colleague tells you that you should use the **F1-score** as a better measure for this model. Do you agree or disagree? Why? Answer in 3 lines only.

Answer — Disagree.

The F1-score is a **classification** measure: it combines precision and recall, which count how many predicted labels are right. Here the target is a **number**, the discovery year, so this is **regression**. There are no labels to be "right" or "wrong", only predictions that are closer to or farther from the true year. Use a regression measure: **R^2** (as in Question 8), or an error in years such as **RMSE** or **MAE**, which are easier to explain to a business audience ("we are off by about N years on average").